

Interaktive Lernplattform für Grundlagen Informatik

Projektarbeit – SS 26

31. Juli 2026

Studenten:	Markus Nöth	markus.noeth@study.thws.de	5123027
	Richard Schröder	richard.schroeder@study.thws.de	5123042
	Enrico Ebert	enrico.ebert@study.thws.de	5123098
	Duy Tan Pham	duytan.pham@study.thws.de	5123063
	Kristian Popov	kristian.popov@study.thws.de	5123029

Dozent: Prof. Dr. Peter Braun

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Problemstellung (Author) . . . 1
1.2 Zielsetzung der Arbeit (Author) . . . 1
1.3 Methodik (Author) . . . 1

2 Übergreifende Architektur & Systemdesign **1**
2.1 Technologiestack (Author) . . . 1
2.2 Modulare Softwarearchitektur (Author) . . . 1
2.3 Unified UI-Konzept (Author) . . . 1
2.4 Simulator-Architektur (Author) . . . 1
2.5 Datenmodellierung (Author) . . . 1

3 Einsatz von Künstlicher Intelligenz im Projekt **1**
3.1 KI Reflexion (Markus Nöth) . . . 1
3.1.1 KI als Co-Developer (Markus Nöth) . . . 2
3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth) . . . 2
3.1.3 Kritische Reflexion (Markus Nöth) . . . 2
3.2 KI Reflexion (Richard Schröder) . . . 2
3.2.1 KI als Co-Developer (Richard Schröder) . . . 2
3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder) . . . 2
3.2.3 Kritische Reflexion (Richard Schröder) . . . 2
3.3 KI Reflexion (Enrico Ebert) . . . 2
3.3.1 KI als Co-Developer (Enrico Ebert) . . . 2
3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert) . . . 2
3.3.3 Kritische Reflexion (Enrico Ebert) . . . 2
3.4 KI Reflexion (Duy Tan Pham) . . . 2
3.4.1 KI als Co-Developer (Duy Tan Pham) . . . 2
3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham) . . . 3
3.4.3 Kritische Reflexion (Duy Tan Pham) . . . 3
3.5 KI Reflexion (Kristian Popov) . . . 3
3.5.1 KI als Co-Developer (Kristian Popov) . . . 3
3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov) . . . 4
3.5.3 Kritische Reflexion (Kristian Popov) . . . 4

4 Designentscheidungen der Fachmodule **5**
4.1 Turing Maschine (Author) . . . 5
4.1.1 Entscheidungen in der Architektur (Author) . . . 5
4.1.2 Entscheidungen im UI (Author) . . . 5
4.1.3 Andere Entscheidungen (Author) . . . 5
4.2 Endliche Automaten (Author) . . . 5
4.2.1 Entscheidungen in der Architektur (Author) . . . 5
4.2.2 Entscheidungen im UI (Author) . . . 5
4.2.3 Andere Entscheidungen (Author) . . . 5
4.3 Reguläre Sprachen (Author) . . . 5
4.3.1 Entscheidungen in der Architektur (Author) . . . 5
4.3.2 Entscheidungen im UI (Author) . . . 5
4.3.3 Andere Entscheidungen (Author) . . . 5
4.4 Zahlensysteme (Author) . . . 5
4.4.1 Entscheidungen in der Architektur (Author) . . . 5

4.4.2 Entscheidungen im UI (Author) . . . 5
4.4.3 Andere Entscheidungen (Author) . . . 5
4.5 Moore & Mealy Automaten (Author) . . . 5
4.5.1 Entscheidungen in der Architektur (Author) . . . 5
4.5.2 Entscheidungen im UI (Author) . . . 5
4.5.3 Andere Entscheidungen (Author) . . . 5
4.6 Disjunktive Normalform (Author) . . . 6
4.6.1 Entscheidungen in der Architektur (Author) . . . 6
4.6.2 Entscheidungen im UI (Author) . . . 6
4.6.3 Andere Entscheidungen (Author) . . . 6
4.7 Encoding (Author) . . . 6
4.7.1 Entscheidungen in der Architektur (Author) . . . 6
4.7.2 Entscheidungen im UI (Author) . . . 6
4.7.3 Andere Entscheidungen (Author) . . . 6
4.8 Java (Author) . . . 6
4.8.1 Entscheidungen in der Architektur (Author) . . . 6
4.8.2 Entscheidungen im UI (Author) . . . 6
4.8.3 Andere Entscheidungen (Author) . . . 6

5 Evaluation & Projekterfahrung 6
5.1 Qualitätssicherung (Author) . . . 6
5.2 Abgleich mit dem Lastenheft (Author) . . . 6
5.3 Lessons Learned (Author) . . . 6
5.4 Design-Iterationen (Author) . . . 6

6 Fazit & Ausblick 7
6.1 Zusammenfassung der Projektergebnisse (Author) . . . 7
6.2 Perspektiven (Author) . . . 7

Anhang i
Abkürzungsverzeichnis . . . i
Literatur . . . i

Abbildungsverzeichnis

Tabellenverzeichnis

1 Einleitung

Eine Arbeit von Technische Hochschule Würzburg-Schweinfurt (THWS) Studenten.

1.1 Problemstellung (Author)

Die Hürde der Abstraktion in Grundlagen Informatik

1.2 Zielsetzung der Arbeit (Author)

Didaktische Aufbereitung komplexer Modelle durch interaktive Simulationen

1.3 Methodik (Author)

Vorgehensweise im Team und Rollenverteilung

2 Übergreifende Architektur & Systemdesign

2.1 Technologiestack (Author)

React 19, TypeScript, Vite, Tailwind CSS 4, shadcn/ui, Zustand, i18next, Lucide Icons

2.2 Modulare Softwarearchitektur (Author)

Single Source of Truth (`moduleCatalog.tsx`), lazy Loading, Shared Topic Pattern – Sicherstellung der Erweiterbarkeit für neue Themen

2.3 Unified UI-Konzept (Author)

Wiederverwendbare Shared Components, konsistentes Designsystem (CSS-Variablen, Dark Mode, moduleigene Akzentfarben, Geist Font)

2.4 Simulator-Architektur (Author)

Das `ISimulator<TConfig, TState>`-Interface und der generische `useSimulationEngine`-Hook als Grundlage aller schrittweisen Simulationen

2.5 Datenmodellierung (Author)

Import/Export über `ISimulatorTransferCodec` mit Base64-Kodierung für Vorlesungskompatibilität

3 Einsatz von Künstlicher Intelligenz im Projekt

3.1 KI Reflexion (Markus Nöth)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.1.1 KI als Co-Developer (Markus Nöth)

Reflektieren und so...

3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth)

Reflektieren und so...

3.1.3 Kritische Reflexion (Markus Nöth)

Reflektieren und so...

3.2 KI Reflexion (Richard Schröder)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.2.1 KI als Co-Developer (Richard Schröder)

Reflektieren und so...

3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder)

Reflektieren und so...

3.2.3 Kritische Reflexion (Richard Schröder)

Reflektieren und so...

3.3 KI Reflexion (Enrico Ebert)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.3.1 KI als Co-Developer (Enrico Ebert)

Reflektieren und so...

3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert)

Reflektieren und so...

3.3.3 Kritische Reflexion (Enrico Ebert)

Reflektieren und so...

3.4 KI Reflexion (Duy Tan Pham)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.4.1 KI als Co-Developer (Duy Tan Pham)

Reflektieren und so...

3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham)

Reflektieren und so...

3.4.3 Kritische Reflexion (Duy Tan Pham)

Reflektieren und so...

3.5 KI Reflexion (Kristian Popov)

Während der Entwicklung des Projekts lag mein Schwerpunkt auf den Modulen *Endliche Automaten* und *Reguläre Sprachen*.

In dieser Zeit habe ich verschiedene KI-Modelle ausprobiert, darunter Gemini 3.1 Pro, die kostenlose Version von ChatGPT sowie Claude Opus 4.8 und später Claude Opus 5.

Dabei ist mir aufgefallen, dass jedes Modell unterschiedliche Stärken besitzt. Manche eigneten sich besser für das Erklären theoretischer Konzepte, andere lieferten präzisere Unterstützung bei der Implementierung oder beim Auffinden von Fehlern im Code. Im Laufe des Projekts habe ich überwiegend Claude verwendet, da ich persönlich die Antworten bei Programmieraufgaben als konsistenter empfunden habe und weniger Korrekturen vornehmen musste als bei den anderen Modellen.

Die KI wurde hauptsächlich als Unterstützung bei der Entwicklung eingesetzt, beispielsweise zum Diskutieren von Lösungsansätzen, zur Fehlersuche, zum Verständnis komplexer Zusammenhänge sowie zur Überarbeitung einzelner Codeabschnitte. Alle vorgeschlagenen Lösungen wurden jedoch eigenständig geprüft, getestet und an die Anforderungen des Projekts angepasst. Die Verantwortung für die technischen Entscheidungen und die finale Implementierung lag jederzeit bei mir.

3.5.1 KI als Co-Developer (Kristian Popov)

Den größten Nutzen brachte die KI dort, wo Struktur wichtiger war als Fachlogik. Das Aufsetzen wiederkehrender React-Komponenten, das Ableiten von TypeScript-Typen aus einer bestehenden Datenstruktur und das Übertragen eines bereits etablierten Musters auf ein weiteres Modul liefen deutlich schneller als in reiner Handarbeit. Ein konkretes Beispiel ist der Übungsbetrieb: Nachdem die Übungen für reguläre Sprachen mit automatischer Lösungskontrolle und gespeichertem Lernfortschritt einmal entworfen und manuell abgesichert waren, ließ sich dasselbe Muster mit KI-Unterstützung auf die endlichen Automaten übertragen.

Ebenso hilfreich war die KI bei Arbeiten am Rand der eigentlichen Implementierung: beim Formulieren verständlicher Fehlermeldungen, beim Pflegen der i18n-Schlüssel für beide Sprachfassungen und beim Aufspüren von Stellen, an denen ein neu eingeführter Zustand noch nicht in allen betroffenen Komponenten berücksichtigt war. Für die Verifikation am laufenden System wurde zusätzlich ein Playwright-MCP-Server eingebunden, sodass Selektoren und sichtbares Verhalten direkt an der Anwendung geprüft und nicht aus dem Quelltext erraten wurden.

Entscheidend war dabei die Rollenverteilung: Die fachliche Modellierung – also die Frage, was ein Automat in einem bestimmten Schritt tun *darf* – blieb durchgehend in meiner Verantwortung. Die KI hat diesen Entwurf umgesetzt und verbreitert, ihn aber nicht festgelegt.

3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov)

Für den Übungsbetrieb der Module *Endliche Automaten* und *Reguläre Sprachen* war nicht allein die Simulation relevant, sondern auch der Aufgabenbestand selbst. Eine Übung benötigt eine Aufgabenstellung, eine hinterlegte Musterlösung und eine automatische Kontrolle der Eingabe; erst dadurch entsteht ein Lernangebot, das ohne Betreuung funktioniert.

Die KI diente hier vor allem der Variantenbildung: Ausgehend von einer manuell entworfenen Referenzaufgabe ließen sich weitere Aufgaben desselben Typs mit anderem Alphabet, anderer Wortlänge oder anderem Schwierigkeitsgrad erzeugen. Die eigentliche Korrektheitsprüfung wurde der KI hingegen bewusst nicht überlassen, sondern deterministisch implementiert: Eine Studierendeneingabe wird gegen das tatsächliche Verhalten des Automaten beziehungsweise der Grammatik geprüft und nicht gegen eine von einem Sprachmodell erzeugte Erwartung. Generierte Aufgaben mussten daher ausnahmslos gegen die Simulation gegengeprüft werden, bevor sie in den Aufgabenkatalog aufgenommen wurden.

3.5.3 Kritische Reflexion (Kristian Popov)

Die deutlichsten Grenzen zeigten sich genau dort, wo das Fachgebiet formale Präzision verlangt. Sprachmodelle erzeugen plausibel wirkenden Code, und bei formalen Sprachen ist „plausibel“ nicht dasselbe wie „korrekt“. Mehrere Korrekturen in meiner Commit-Historie lassen sich unmittelbar darauf zurückführen: die Verwechslung von Kleene-Stern und Kleene-Plus in der Sprachbeschreibung, eine fehlerhafte Wortaufzählung, Grammatiken, die entgegen der Anforderung nicht rechtslinear waren, sowie ein Automatenlauf, der beim Erreichen einer Sackgasse die aktiven Zustände nicht zurücksetzte. Solche Fehler sind beim Überfliegen schwer zu erkennen, weil die Ausgabe formal wohlgeformt aussieht – sie fallen erst beim systematischen Durchspielen konkreter Eingaben auf.

Ein zweites wiederkehrendes Muster war strukturelle Redundanz. Da sich die Module *Endliche Automaten* und *Reguläre Sprachen* stark ähneln, neigte die KI dazu, bestehende Implementierungen zu duplizieren statt zu abstrahieren. Das führte zu Simulator- und Übersichtsseiten, die sich zwischen den Modulen kaum unterschieden, sowie zu Funktionalität, die zwar erzeugt, aber nie in die Oberfläche eingebunden wurde und später wieder entfernt werden musste. Ohne bewusste Nachbearbeitung entsteht auf diesem Weg Code, der zwar funktioniert, die gemeinsame Architektur des Projekts jedoch unterläuft.

Hinzu kam die Kontextfenster-Problematik: Bei Änderungen, die beide Module zugleich betrafen – etwa der einheitlichen Gruppierung von Beispielen und Übungen nach Kategorien – reichte der Überblick des Modells nicht aus, um alle betroffenen Stellen zuverlässig zu erfassen. Solche Änderungen mussten in kleinere, jeweils überschaubare Schritte zerlegt werden.

Als Fazit bleibt: KI hat die Entwicklungsgeschwindigkeit spürbar erhöht, vor allem beim Verbreitern bereits verstandener Lösungen. Für den fachlich kritischen Kern – die korrekte Abbildung formaler Modelle – blieb sie ein Vorschlagsgeber, dessen Ergebnisse ausnahmslos gegen die Theorie und gegen die laufende Anwendung verifiziert werden mussten. Der eingesparte Implementierungsaufwand verschiebt sich damit teilweise in einen erhöhten Prüfaufwand.

Ein Ausblick bleibt offen: Gegen Ende des Projekts wurde mit Sakana Fugu ein System vorgestellt, das als Orchestrator verschiedene KI-Modelle je nach Aufgabenstellung auswählt [1]. Da es erst kurz vor Abschluss meiner Arbeit veröffentlicht wurde, konnte ich es nicht mehr praktisch in das Projekt integrieren und evaluieren. Gerade für den hier beschriebenen Zielkonflikt – schnelle Umsetzung bei gleichzeitig erhöhtem Prüfaufwand – wäre eine aufgabenabhängige Modellwahl jedoch ein naheliegender nächster Schritt.

4 Designentscheidungen der Fachmodule

4.1 Turing Maschine (Author)

4.1.1 Entscheidungen in der Architektur (Author)

4.1.2 Entscheidungen im UI (Author)

4.1.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.2 Endliche Automaten (Author)

4.2.1 Entscheidungen in der Architektur (Author)

4.2.2 Entscheidungen im UI (Author)

4.2.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.3 Reguläre Sprachen (Author)

4.3.1 Entscheidungen in der Architektur (Author)

4.3.2 Entscheidungen im UI (Author)

4.3.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.4 Zahlensysteme (Author)

4.4.1 Entscheidungen in der Architektur (Author)

4.4.2 Entscheidungen im UI (Author)

4.4.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.5 Moore & Mealy Automaten (Author)

4.5.1 Entscheidungen in der Architektur (Author)

4.5.2 Entscheidungen im UI (Author)

4.5.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.6 Disjunktive Normalform (Author)

4.6.1 Entscheidungen in der Architektur (Author)

4.6.2 Entscheidungen im UI (Author)

4.6.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.7 Encoding (Author)

4.7.1 Entscheidungen in der Architektur (Author)

4.7.2 Entscheidungen im UI (Author)

4.7.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.8 Java (Author)

4.8.1 Entscheidungen in der Architektur (Author)

4.8.2 Entscheidungen im UI (Author)

4.8.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

5 Evaluation & Projekterfahrung

5.1 Qualitätssicherung (Author)

CI/CD-Pipeline (GitLab Runner auf Raspberry Pi), automatisiertes Linting, Formatierung (Prettier), Unit-Tests (Vitest) und E2E-Tests (Playwright)

5.2 Abgleich mit dem Lastenheft (Author)

Wurden die Qualitätsziele (Performance, Usability, Barrierefreiheit durch Dark Mode und i18n) erreicht?

5.3 Lessons Learned (Author)

Herausforderungen in der Team-Kollaboration (Git-Workflow, Merge-Requests, geteilte Architektur) und technische Hürden

5.4 Design-Iterationen (Author)

Welche Entscheidungen mussten im Prozess revidiert werden?

6 Fazit & Ausblick

6.1 Zusammenfassung der Projektergebnisse (Author)

Sieben interaktive Fachmodule in zwei Lernbereichen

6.2 Perspektiven (Author)

Gamification (Badges/Punkte), LMS-Anbindung (Moodle/ILIAS) und Mobile Learning (responsive Optimierung)

Anhang

Abkürzungsverzeichnis

THWS Technische Hochschule Würzburg-Schweinfurt; Hochschule aus Würzburg und Schweinfurt (Bayern) [2]

Literatur

[1] Sakana AI. „Sakana Fugu - Multi-agent System as A Model," besucht am 31. Juli 2026. Adresse: `https://sakana.ai/fugu/`

[2] Technische Hochschule Würzburg-Schweinfurt. „THWS - Technische Hochschule Würzburg-Schweinfurt," besucht am 14. Juni 2026. Adresse: `https://thws.de`